

백엔드 트랙 — JVM 파운데이션·신뢰성 → 스포츠북 캡스톤

이 트랙은 무엇인가. 이 코퍼스의 주력(★). Java 21·Spring으로 돈·먹등·트랜잭션·동시성·보안 이라는 백엔드의 심장을 손으로 쌓고, 그것을 분산 시스템 캡스톤 sportsbook (해외 스포츠북 백엔드)으로 통합한다. 코드 <i>재ഖ</i>이 아니라 <i>판단력</i> — L3만 백지에서 다시 만들고 나머지는 빠르게 통과한다.
언제 시작하나. 42 트랙에서 M0(linux-admin) 작업·커밋 규율을 이미 끝낸 뒤 분기해 들어온다(이 트랙은 그 규율 게이트를 다시 다루지 않는다 — 필요한 커밋 규율은 아래 <공통 어휘>에 정리해 둔다).
읽는 법. 위에서 아래로 한 프로젝트씩 헤쳐나가면 완주된다. 각 블록은 자족적이다 — 그 프로젝트의 노트 순서·재구성 방법·L3·검증·합정·완료 바가 블록 안에 다 있다. 이 문서 하나로 트랙을 완주하도록 썼다.
딱 한 번만 외우는 공통 어휘 (이후 각 블록은 이 말만 쓴다): - L3 = 돈·먹등·트랜잭션·동시성·보안·outbox 등 <i>폭발 변경</i> → <i>결정</i>·<i>답지</i>·<i>덮고</i>·<i>백지</i>에서 코드·이유를 재구성하고 실패 모드까지 방어돼야 “끝”. 이 코퍼스의 심장이라 2회독한다. - L2 = 구조·흐름·책임 분리(필처 설명되면 충분). L1 = 보일러플레이트·CRUD·설정(읽고 통과). - 핵심 동작 = 이 트랙의 리듬은 대부분 triplet(2A)다 — ① <code>docs: frame</code> (문제 계약 문서)을 읽고 → ② <i>거기서 멈춰</i>·답지·덮고, 계약만 보고 직접 구현 → ③ <code>feat</code>·답지 펼쳐 diff → ④ <code>test</code> 문서로 검증 모델 익히기. 답지 끝 <code>## 기억/설명 Level</code>에 L3/L2/L1이 적혀 있으니 들어가기 전에 본다. - 커밋 규율 (학습 산출물을 커밋할 때 — 이 트랙 전 레포 공통): 제목은 영어 명령형 <code><type>(<scope>): ...</code> (type ∈ feat·fix·docs·test·refactor·chore·build·ci·perf), 본문은 한국어 [근거] 왜 / [변경] 무엇 / [검증] 어떻게 (잡일·merge는 면제), AI 공동저자 트레일러 없음. 날짜는 KST·author==committer·근무시간 09:00-21:59·분초 랜덤·중복 금지. dual-form 문서는 답지 <code>docs(commits)</code> / 문제지 <code>docs(practice)</code> 스코프로 나눠 바뀐 파일만 add(→A 금지). 후속 작업은 새 phase로 답지 append.

전체 경로:

backend-foundations-training → backend-reliability-training → container-stack → ★sportsbook(캡스톤) (①·2A) (⑤·2A) (②·2B) (9 레포 순서 고정)
이 트랙 내부는 순차다. sportsbook 은 foundations + reliability + container-stack 셋이 전부 끝난 뒤에야 진입한다(자세한 의존은 맨 아래 “병렬성”).

1. backend-foundations-training — JVM 대장(구현 기초)

무엇/왜. Java 21 + Spring Boot 구현 기초를 17개 exercise 슬롯으로 익힌다 — HTTP·JSON → CRUD → validation → JPA/H2 영속성 → 관계 → 테스트 슬라이스 → 세션 인증 → Security → pagination → 파일 업로드, 그리고 같은 패턴을 Go 표준 라이브러리로 재현하는 비교 트랙. JVM 트랙의 대장이자 코퍼스의 심장 입구. 이 트랙에서 얻는 것. Spring으로 “요청→검증→트랜잭션→응답”을 짜는 손과, 트랜잭션·먹등·보안의 첫 L3.
--

장르·리듬. 장르 ④ 따라만들기(exercise 카탈로그) · **triplet(2A)**: 각 과제가 `docs: frame` (계약) → `feat: implement` → `test: cover` 3커밋. 답지 75 / 문제지 44. docs/notes/ 보유.

노트 읽는 순서 (**backend-foundations-training/notes/** , **목음**별). - 언어: `java-21` → `jdk-21` - 빌드: `gradle` → `spring-boot-gradle-plugin` - 코어: `spring-boot` → `jakarta-servlet-api` → `spring-web-mvc` - 기능: `spring-boot-starter-validation` → `spring-boot-starter-security` → `spring-data-commons-pagination` - 테스트: `spring-test` → `junit5` → `mockito` → `spring-security-test` → `testcontainers` - Go 비교: `go-toolchain` → `internal-go-cli-checkdocs`

처음 보는 스택(Spring·JPA)은 1부 개념부터 깊게, `reference-impl/` 을 한 번 빌드한다.

재구성·커밋 활용법. 단계로 본다 — 000~002 계약·골격(17 슬롯 선등록, L1~L2) - **004~042 Spring spine** — 각 슬롯이 `frame=impl+test` triplet. **L3 슬롯**: 005(Spring MVC), 009(CRUD 상태 소유권), 013(validation), 017(JPA/H2), 021(관계), 024(테스트 슬라이스), 029(수동 세션), 033(Security session), 037(pagination), 041(multipart). 각 슬롯에서 `frame`을 읽고 **덮고 직접 구현** → `make test-spring-exercise EX=<슬롯> green` → 답지 diff. - 044~054 캡스톤(board/commerce/task에 spine 결합) - **056~070 bridge — reliability** 직전 단계: **PostgreSQL profile** · **transaction boundary** · **idempotent create** · **async job (L3 핵심)**. 057·060·061·064·068·069를 특히 손으로. - 072~089 Go sub-track(표준 라이브러리로 같은 패턴 4개 재현) · 091~100 CI·검증 불인.

L3 — 손으로 백지 재구성할 것. **transaction boundary**(어디서 열고 닫나), **idempotent create** (같은 요청 두 번 = 한 번 효과), CRUD 상태 소유권, validation 실패 분리, Security 세션 경계. bridge(056~070)가 reliability로 가는 다리이니 여기 L3는 빼먹지 마라.

검증. `make test` / `make test-spring` / `make test-go` / `make check-docs`, 슬롯 단위는 `make test-spring-exercise EX=<슬롯>`.

합정. Spring Security 의존성은 해당 슬롯에만 격리(초반 HTTP/CRUD에 인증 필터 끼여들지 않게). `bootJar off`·`jar on`(메인 클래스 없는 초기 슬롯도 테스트로 진행). Go sub-track은 *비교 학습*이라 같은 레포에 둔다 — 건너뛰면 가지 반감. **frame 커밋이 문제 계약을 먼저 세우는 설계** — 구현부터 보지 마라.

완료 바 → **다음으로**. `transaction boundary-idempotent create`를 **백지로** 방어되면 reliability로. bf는 *트랜잭션·보안* 중심, br은 *동시성·롤백* 중심으로 역할이 갈린다.

커밋 메모. dual-form 스코프: 답지 `docs(commits)` / 문제지 `docs(practice)` 분리. triplet의 `frame`은 순수 계약 문서라 답지만(문제지 없음) — 실제 있는 `feat` 커밋만 문제지를 가진다.

2. backend-reliability-training — 신뢰성 결정 패턴

무엇/왜. 하나의 앱이 아니라 신뢰성 결정 패턴을 회귀 테스트로 봉인하는 exercise 묶음 — 먹등성, outbox, transaction race, retry, cache invalidation, rate limiter, saga 등. JPA/transaction/Redis 위에서 “올바른 결정”을 코드로 못 박는다. (+ Go·Kotlin DSL.) 이 트랙에서 얻는 것. 동시성·롤백·outbox·캐시 정합 — sportsbook 캡스톤에서 그대로 재사용할 L3.
--

장르·리듬. 장르 ⑤ 작업규율(서비스·레벨 테스트·Testcontainers 기반, HTTP controller 없음) · **2A**: README 계약 → 구현 → 테스트. 답지 68 / 문제지 39.

노트 읽는 순서 (**backend-reliability-training/notes/** , **목음**별). - 빌드: `gradle-kotlin-dsl` - 영속/트랜잭션: `jakarta-persistence` → `spring-data-jpa` → `h2-database` → `spring-transaction` - 캐시: `redis` → `spring-data-redis` - 보조: `slf4j` → `assertj` - Go: `go-language` → `go-net-http`

재구성·커밋 활용법. 구간으로 본다(번호는 `docs/commits/README.md` 의 phase 경계로 확인) — 000~001 스케폴드(Gradle multi-project·Compose local infra) - **003~015 essential ①**: `signup invariants` · `idempotency key` · `transaction race`(비관 락) — **L3 - 017~027 essential ②**: `cursor pagination` · `rate limiter`(token bucket·key isolation) — **L3 - 029~038**: `job queue retry` 상태 전이 · `cache invalidation`(hit/miss) — **L3 - 040~048 advanced ①**: `outbox pattern`(atomic record) · `authorization policy matrix` — **L3 - 050~063 advanced ②**: `file upload safety` · `observability tracing` · `failure injection` — **L3 - 067~082 Go 보충**(token bucket·worker pool·HTTP timeout·minimal idempotency) · 084·086 CI·SRE/domain drill.

각 구간 README 계약을 먼저 읽고 **덮고 service 테스트가 green이 되게** 구현한 뒤 답지 diff.

L3 — 손으로 백지 재구성할 것. `idempotency key`(중복 제거), `transaction race` + **비관 락**, `outbox`(상태 변경과 이벤트 발행의 원자성), `retry` 상태 전이, `cache invalidation` 정합, token bucket rate limit의 key 격리, `authorization policy decision`.

검증. `make test` / `make test-spring` / `make test-go`, 단위는 `make test-spring EX=<name>`. 동시성·unique constraint 검증의 표준은 **Testcontainers PostgreSQL**(H2 풀백 아님).

합정. exercise는 application이 아니라 **testable module**(`bootJar off`). 외부 module 금지 — distributed rate limit·circuit breaker를 **표준 라이브러리로** 쓴다(라이브러리 가져다 쓰면 학습 0). “단일 제품 path 아님” — 완성된 앱을 기대하지 말고 패턴 하나하나를 봉인.

완료 바 → **다음으로**. 먹등·트랜잭션 레이스·outbox를 **백지로** 방어되면 인프라(container-stack)로. 이 셋은 sportsbook의 `wallet-betting-settlement`에서 그대로 다시 나온다.

3. container-stack — 인프라/운영(sportsbook 선행)

무엇/왜. Docker·Compose·Nginx·POSIX shell로 42 Inception식 WordPress 스택(nginx TLS 진입 + PHP-FPM + MariaDB)을 조립한다. 만드는 법이 아니라 왜 이 정책인가(헬스체크 체인·secret 분리·진입 경계)가 핵심. sportsbook을 분산 시스템으로 띄우기 전 인프라 사고의 선행이다. 이 트랙에서 얻는 것. Compose 네트워킹·healthcheck·secret·DB 초기화를 <i>왜 이렇게</i> 두는지 방어하는 힘.
--

장르·리듬. 장르 ② **절차·설정형** · 순차(2B). 답지 19 / 문제지 18(거의 1:1) — **코드형처럼 덮고 설정을 재구성**하되, 재정은 정책 검증기 + 실제 기동.

노트 읽는 순서 (**container-stack/notes/** , **목음**별). - 셸: `posix-shell` → `apt` → `curl` → `gosu` - 컨테이너: `docker` → `dockerfile` → `yaml` → `docker-compose` - 웹서버: `nginx-config` → `openssl` → `cgi-fcgi` → `php-fpm-pool` → `wordpress` → `wp-cli` - DB: `mariadb-install-db` → `mariadb-config` → `mysqladmin`

재구성·커밋 활용법. 000~001(Compose 서비스 경계·환경변수 템플릿: **nginx 443만 공개**, **DB/WP 내부**, fail-fast) → 002~004(MariaDB: 이미지→설정→초기화 endpoint, L3: **persistent volume** **관별·bootstrap SQL**) → 005~007(WordPress: PHP-FPM→pool→wp-cli bootstrap, L3: **DB 대가·재실행 안정성**) → 008~009(nginx TLS·FastCGI) → **010~012(readiness·healthcheck 체인·공유 volume·Compose secrets, L3)** → 013~018(정적 validator·compose config·HTTPS smoke). 설정을 덮고 다시 쓰며 “왜 이 디렉티브”를 방어한다.

L3 — 손으로 백지 재구성할 것. 구현이 아니라 **정책**: 헬스체크/readiness 체인, secret 분리 (`.env` · `secrets/*.txt`·Git 미추적), 진입 경계(443만 공개), Compose 네트워킹, **wp-cli idempotent 설치** (volume 재실행에도 안정), `${VAR:?set VAR}`: fail-fast.

검증. `make test` (정적 검증 우선, **Docker daemon 불필요**) / `make smoke` (HTTPS /healthz) / `make build` · `up` · `down` · `clean` · `fclean`. (통합 검증은 Docker 있으면 실제 기동까지.)

합정. 첫 커밋이 구현 없이 README·골격만 고정 — 이후 작업이 놓일 자리를 먼저 만든다. 자체 서명 인증서를 컨테이너 *안에서* 만들지 않는다(외부 신뢰 체인 아님). 통합 검증엔 Docker 필요(없으면 정적까지).

완료 바 → **다음으로**. 백지에서 헬스체크 체인·secret 분리·진입 경계를 *왜 이렇게*인지 방어되면 — 이제 **캡스톤**으로. 여기서 익힌 Compose 조립이 sportsbook의 9레포 통합 기동의 기반이다.

4. ★ sportsbook/ — 캡스톤: 해외 스포츠북 백엔드(9 레포 분산 시스템)

무엇/왜. 온라인 스포츠북 백엔드. 9개 독립 git 레포(서비스 7 + gateway + orchestration)로, 베팅 접수는 동기 orchestration, 정산은 비동기 Saga + Outbox로 도는 분산 시스템이다. 두 갈래로 학습한다 — (A) 코어 = 종합 적용(트랙 앞의 139개 노트 <i>재사용</i>, 새 학습 아님). (B) 신규 = 분산·운영 스택(Tier-1 8노트). <code>sportsbook/</code> 은 레포가 아니라 여러 독립 레포를 담은 폴더다 — 하위 각 레포가 작업 단위(<code>find</code>로 잡힌 <code>.git</code> 기준). 이 트랙에서 얻는 것. 돈·먹등·트랜잭션·outbox·분산을 하나의 서사로 통합한 커리어 산출물.

진입 체크. foundations + reliability + container-stack **완료 시 자연스럽다**. 못 채웠으면 돌아가라.

장르·리듬. 장르 ①(마이크로서비스 통합 재구성) · 각 서비스는 자기 `docs/commits/` 로 **\$2 루프** 그대로.

신규 학습 — **Tier-1 8노트 읽는 순서** (**sportsbook/orchestration/notes/**). 이게 139개에 없는 횡단 계층이다. 서비스 들어가기 전 해당 노트 1편을 선행한다:

maven → spring-kafka → avro → flyway → resilience4j(bucket4j 포함) → spring-cloud-gateway → oauth2-resource-server → observability

서비스 순서 고정 + 각 책임. (의존 방향의 역순 = 호출당하는 leaf부터 빌드) 1. **shared-protocol** — Avro 이벤트 스키마 + value object(Money/Odds/ID) + enum. 모든 서비스가 의존. 순수 계약이라 *재요도할 설계가 핵심인 schema*만 얹게 연승. 2. **wallet-service** — **double-entry ledger**: 잔고 + 모든 자금 이동, 먹등 debit/credit/forfeit. (L3 밀집) 3. **risk-service** — per-user/market 한도·패턴 탐지. betting critical path라 **p99 < 30ms 지연 예산**. 4. **odds-feed-service** — odds ingress → Kafka publish + Redis write-through. 5. **betting-service** — **베팅 접수(동기 orchestration)**: slip 검증 → risk·wallet 동기 호출 → transaction-outbox. (L3 밀집) 6. **settlement-service** — 이벤트 기반 정산: MatchResult → WON/LOST/PUSH/VOID → payout/forfeit. Kafka consumer + **Saga + DLQ + outbox**. (L3 밀집) 7. **gateway** — 유일한 공개 진입점: RS256 JWT, rate limit(Bucket4j+Redis). `/api/v1/*` 라우팅. STOMP. 8. **admin-api** — 운영자 표면(void/refund/limit/market-close + audit). 사용자 gateway와 분리해 blast-radius(사고 폭발 반경) 격리. 9. **orchestration** — 통합 hub: full-stack compose + e2e + chaos + observability overlay.

빠른 신호로 규모를 본다 — wallet 22·betting 28·settlement 25 커밋이 가장 무겁고 L3가 몰린다. shared-protocol 19, risk 13, odds-feed 14, gateway 8, admin 8, orchestration 10.
--

재구성·커밋 활용법. 각 서비스를 **상단 <공통 어휘>의 루프** 그대로(덮고 만들기 → diff) 돌되, 코어(A)는 **기존 노트 재사용**이라 빠르게, 신규(B) Tier-1 + 돈/정산 **L3**는 깊게. 서비스 전환 시 해당 Tier-1 노트 선행 → `docs/commits/` 덮고 재구성 → 답지 diff. 각 서비스 `docs/architecture/decisions` 의 ADR(설계 결정 기록)을 읽어 *왜 이 설계*를 잡는다.

L3 — 손으로 백지 재구성할 것. - **돈** — double-entry ledger 불변식(차변=대변), Money minor-units·BigDecimal 합정. - **먹등** — Idempotency-Key로 debit/credit 중복 차단. - **트랜잭션** · **outbox** — 상태 변경과 이벤트 발행의 원자성(betting·settlement). - **분산** — Saga(접수→정산), partition key, DLQ, Kafka 단일 브로커 **RF=1 합정**(`__consumer_offsets` 생성 불가 → coordinator 부재 → 전 consumer 마비)·호스트 패키징(mavenLocal shared-protocol을 Docker 컴파일러가 못 봐 fat jar 스타이징)·통합 그린 ≠ 단독 그린. - **신뢰성 패턴** — resilience4j(circuit breaker)·Bucket4j(token bucket)·fail-open/closed.

검증. 서비스별 단위/테스트 + 통합:

./scripts/build-all.sh # 호스트 패키징(fat jar 스타이징) docker compose up -d --wait postgres kafka redis docker compose up -d # 전 스택 기동 bash e2e/run-e2e.sh # 통합 e2e (WON payout + LOST forfeit)
--

통합 검증은 Docker 필요(없으면 단위까지, e2e는 답지로).

합정. multi-repo인 이유는 커밋·문서 1:1 무결성 — 상위 폴더를 한 덩어리로 다루지 마라. 스택은 **Java 17 + Spring Boot 3.2**(Kotlin보다 채용 시장 점유율 우선). **접수는 동기·정산은 비동기** (odds 변동성 vs 시간 여유). VI 범위: cash out-in-play·KYC/AML 등은 의도적으로 뺐다 — 놀리지 마라. 각 서비스가 단독으로 green이어서도 **통합 시 인프라·설정·계약 결합이 드러난다** — 그 9개 결합이 학습의 핵심.

완료 바 → **트랙 완주**. 9레포 통합 동작 확인 + Tier-1 8노트 + 돈·먹등·트랜잭션·outbox를 **백지로** 방어 → GitHub push(커리어 산출물). 마지막으로 면접 대비로 아래 축을 백지 설명 세트로 연습한다 — **동시성** · **트랜잭션·정합성** · **먹등성·중복 제거** · **돈 계산**(minor units·BigDecimal 합정) · **분산**·**이벤트**(outbox·partition key·Saga) · **신뢰성 패턴**(circuit breaker·token bucket·fail-open/closed).

커밋 메모. 레포 단위로 커밋·푸시한다(서브에이전트로 나눠도 **레포 세션이 마지막에 한 번**). 상단 <커밋 규율>대로 변경 파일만 add, dual-form 스코프 분리, phase 경계(구현 블록 → 메타·문서 블록) 유지.

병렬성 — 이 트랙의 순서 규칙

- 트랙 내부는 **순차(병렬 불가)**. sportsbook 캡스톤은 foundations + reliability + container-stack이 전부 완료된 뒤 진입한다. 즉 신뢰성을 먼저 끝내야 캡스톤이다 — 신뢰성과 캡스톤을 병행할 수 없다. 신뢰성의 먹등·트랜잭션·outbox가 곧 sportsbook 서비스들의 재료가기 때문이다.
- 트랙 **간(42 / 백 / 프론트)**: 서로 하드 의존이 없어(공통 게이트 M0뿐) 원리상 병렬도 되지만, 이 계획은 42를 먼저 완주하고 백/프론트로 분기하는 단선이다.
- 참고: PostgreSQL은 Spring Data JPA로 충당하므로 pong-pong(Node/TS)을 캡스톤 선행으로 둘 필요는 없다.